



Pós-Graduação em Machine Learning, Deep Learning e Inteligência Artificial
Disciplina: Sistemas Cognitivos e Linguagem Natural

FinNLP

Pipeline de NLP para Atribuição de Performance
e Inteligência de Mercado

Aluno: **Fabio Ferreira Figueiredo**
Professor: **Fernando Guimarães Ferreira**
Instituição: **Instituto Infnet**
Data: **1 de junho de 2026**
Repositório: **github.com/fabioffigueiredo/pd_nlp_finnlp**

Projeto acadêmico — cliente fictício ("Gestão do Fundo"), usado só para dar contexto de negócio
Compliance: corpus 100% público (financial_phrasebank, Hugging Face). Nenhum dado corporativo real foi ingerido.
Nenhuma instituição financeira real é nomeada no projeto.

1. Introdução e contexto do problema

O volume de notícias financeiras disponíveis cresce mais rápido do que a capacidade de leitura manual. Classificar sentimento, identificar temas e mapear relações entre entidades são tarefas repetitivas que se prestam bem à automação por PLN.

O FinNLP é um pipeline end-to-end de Processamento de Linguagem Natural desenvolvido como projeto acadêmico. O sistema ingere notícias financeiras, classifica o sentimento (negativo / neutro / positivo), descobre temas latentes e constrói um grafo de coocorrência entre entidades extraídas do corpus.

1.1 Corpus e justificativa

O corpus de análise é o `financial_phrasebank`, carregado via Hugging Face: 2.264 sentenças em inglês, anotadas por analistas com os rótulos `negative` / `neutral` / `positive`. Escolhi esse dataset por ser o benchmark de referência para sentimento financeiro, por vir rotulado (o que viabiliza a classificação supervisionada) e por ser público e citável. Com 2.264 documentos, passa com folga do mínimo de 1.000 exigido e cobre as três classes de sentimento.

Preciso ser transparente sobre uma escolha: o conteúdo do `financial_phrasebank` é noticiário corporativo nórdico, com forte presença de empresas listadas finlandesas. Isso reaparece nas fases seguintes — no vocabulário dos tópicos e nos hubs do grafo — então prefiro deixar claro desde já em vez de tratá-lo como um corpus de mercado global.

O pipeline também inclui um coletor próprio (`requests` + `BeautifulSoup`) apontado para o portal de economia da Agência Brasil. Na execução, o seletor do portal não retornou artigos, então o scraper entra no projeto como capacidade demonstrada, não como fonte da análise — optei por não inventar um corpus bilíngue que de fato não tinha.

Corpus: https://huggingface.co/datasets/financial_phrasebank (config `sentences_allagree`) | Idioma: inglês (corpus rotulado da análise)

1.2 Guia de Navegação — Rubricas de Avaliação

O pipeline segue a progressão lógica das 5 rubricas exigidas pela disciplina. Cada rubrica corresponde a uma fase do desenvolvimento:

Rubrica 1	Pré-processamento textual (NLTK + spaCy, stemming vs lematização, POS tagging)	Seção 2
Rubrica 2	Representação vetorial e busca semântica (TF-IDF, Word2Vec, cosseno, t-SNE)	Seção 3
Rubrica 3	Modelagem, classificação e tópicos (NB vs SVM, F1, LDA, pyLDAvis, MLflow)	Seção 4
Rubrica 4	NER, extração e grafo de conhecimento (spaCy, RegEx, Levenshtein, NetworkX, SCD2)	Seção 5
Rubrica 5	Visualização, comunicação e reprodutibilidade	Seção 6

2. Rubrica 1 — pré-processamento textual com NLTK e spaCy

Critérios da Rubrica 1 atendidos:

- [v] Pipeline com tokenização, normalização e stopwords customizadas financeiras
- [v] Comparação entre stemming (Porter/RSLP) e lematização (spaCy) com evidências
- [v] POS tagging com spaCy e análise da distribuição gramatical do corpus
- [v] Nuvem de palavras, histograma de comprimento e distribuição POS
- [v] Pipeline modular, reprodutível e coerente com o domínio financeiro

2.1 Decisão técnica: lematização vs stemming

Ao comparar as duas estratégias no corpus financeiro, observei que o stemmer Porter (NLTK) produz radicais irreconhecíveis para jargões essenciais do mercado. Por exemplo: 'acquisitions' → 'acquisit', 'shareholders' → 'sharehold'. A lematização via spaCy preserva a forma dicionarizada ('acquisition', 'shareholder'), crítica para que o TF-IDF reconheça colocações como 'credit default' e 'operating income' na Fase 2.

Adicionalmente, criei uma lista de stopwords customizadas financeiras em inglês e português (ex.: 'said', 'company', 'million', 'empresa', 'resultado') que, sem remoção, dominariam o vocabulário e enviesariam os modelos de tópicos e classificação.

2.2 Nuvem de palavras — vocabulário pós-lematização

Vocabulário Financeiro — Pós-Lematização (FinNLP)



Figura 1: Nuvem de palavras após lematização e remoção de stopwords financeiras.

2.3 Distribuição de comprimento e POS tags

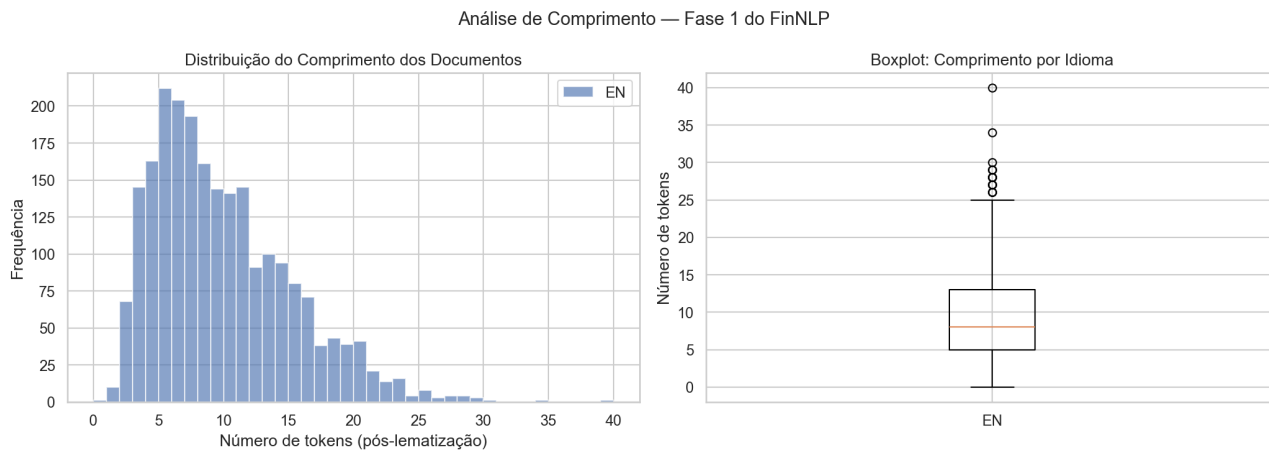


Figura 2: Histograma e boxplot do comprimento dos documentos por idioma.

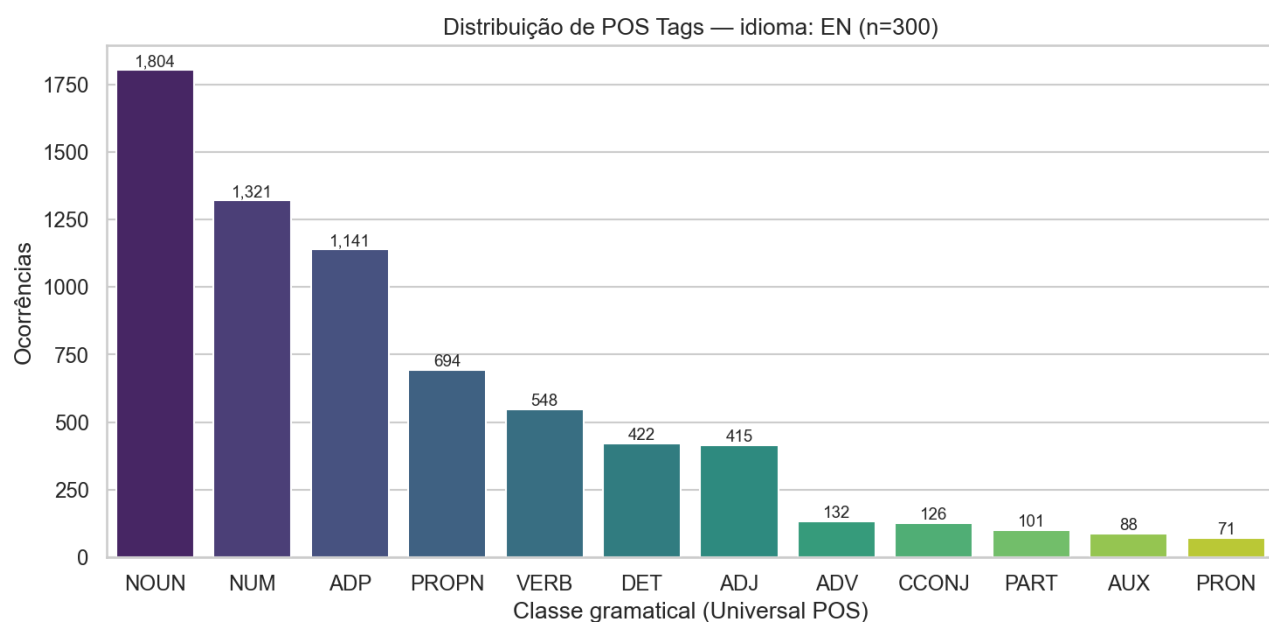


Figura 3: Distribuição de POS tags — alta densidade de NOUN e PROPN confirma domínio financeiro.

A alta frequência de NOUN (substantivos) e PROPN (nomes próprios) é coerente com texto financeiro, onde organizações, índices e instrumentos dominam o vocabulário. Esta característica justifica o foco em NER (Rubrica 4) para extrair valor semântico dessas entidades.

3. Rubrica 2 — representação vetorial e busca semântica

CrITÉRIOS da Rubrica 2 atendidos:

- [v] TF-IDF com unigramas e bigramas (ngram_range=(1,2), 3.787 features)
- [v] Word2Vec treinado no corpus (vector_size=100, window=5)
- [v] Motor de busca por similaridade de cosseno com 3 consultas de performance attribution
- [v] Análise de vizinhos semânticos (Word2Vec) — com leitura crítica das limitações
- [v] Heatmap TF-IDF e visualização t-SNE do espaço vetorial

3.1 TF-IDF com Bigramas

Inclui bigramas (ngram_range=(1,2)) porque colocações financeiras perdem significado quando separadas em unigramas. O efeito aparece no próprio corpus: 'operating profit' é o único bigrama no top-10 por peso TF-IDF médio (posição 8), à frente de unigramas como 'mln' e 'rise'. O heatmap abaixo mostra como os documentos se distinguem pelos termos de maior peso.

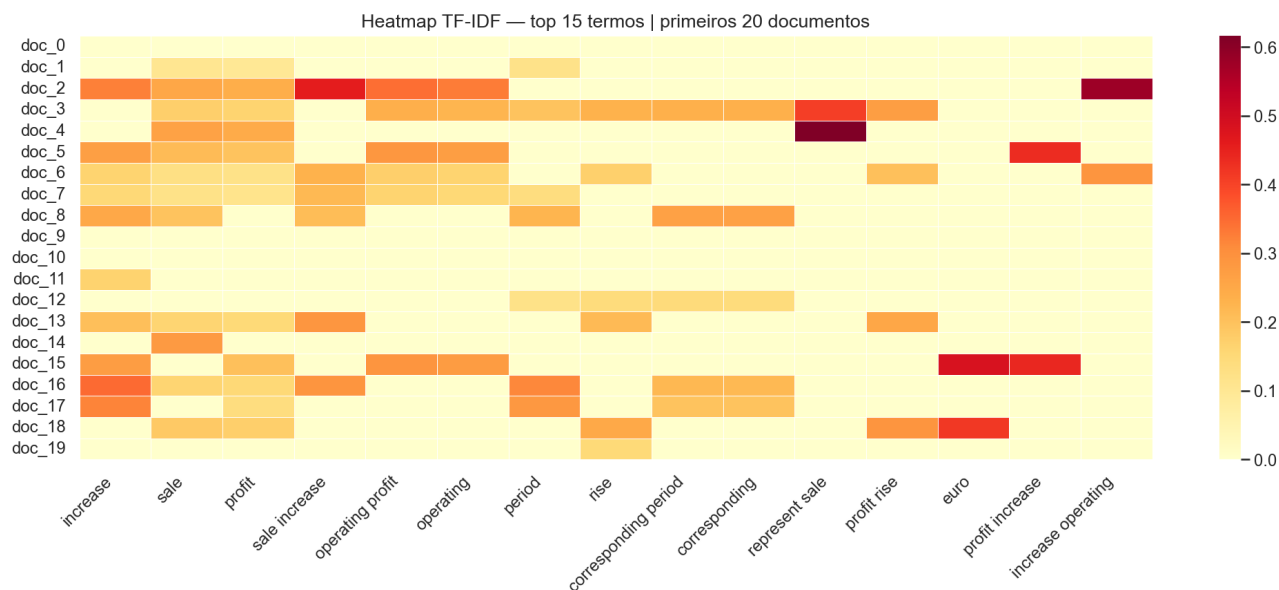


Figura 4: Heatmap TF-IDF — top 15 termos nos primeiros 20 documentos.

3.2 Motor de busca — 3 queries de performance attribution

Implementei um motor de busca que lematiza a consulta, vetoriza com TF-IDF e retorna os documentos mais similares via cosseno. Testei três consultas típicas de análise de textos financeiros, e os scores do melhor resultado de cada uma foram:

- "currency impact retail sector equity allocation" → 0.241 (topo: frase sobre efeito cambial no faturamento)
- "credit risk exposure sovereign default fixed income" → 0.344 (topo: net investment income; aparece também crédito de carbono na UE)
- "earnings growth technology portfolio performance attribution" → 0.353 (topo: documentos de

crecimento e margem)

Os scores são modestos — todos abaixo de 0.4 — e isso é esperado: com sentenças de cerca de 9 tokens e uma matriz TF-IDF 99,7% esparsa, a sobreposição léxica entre query e documento é pequena por construção. Score baixo aqui não significa busca ruim, e sim que o sinal disponível é curto. Mesmo assim, os documentos recuperados são topicamente coerentes com cada consulta. Para subir esse teto, o próximo passo seria indexar por embeddings de sentença.

3.3 Word2Vec e a projeção t-SNE

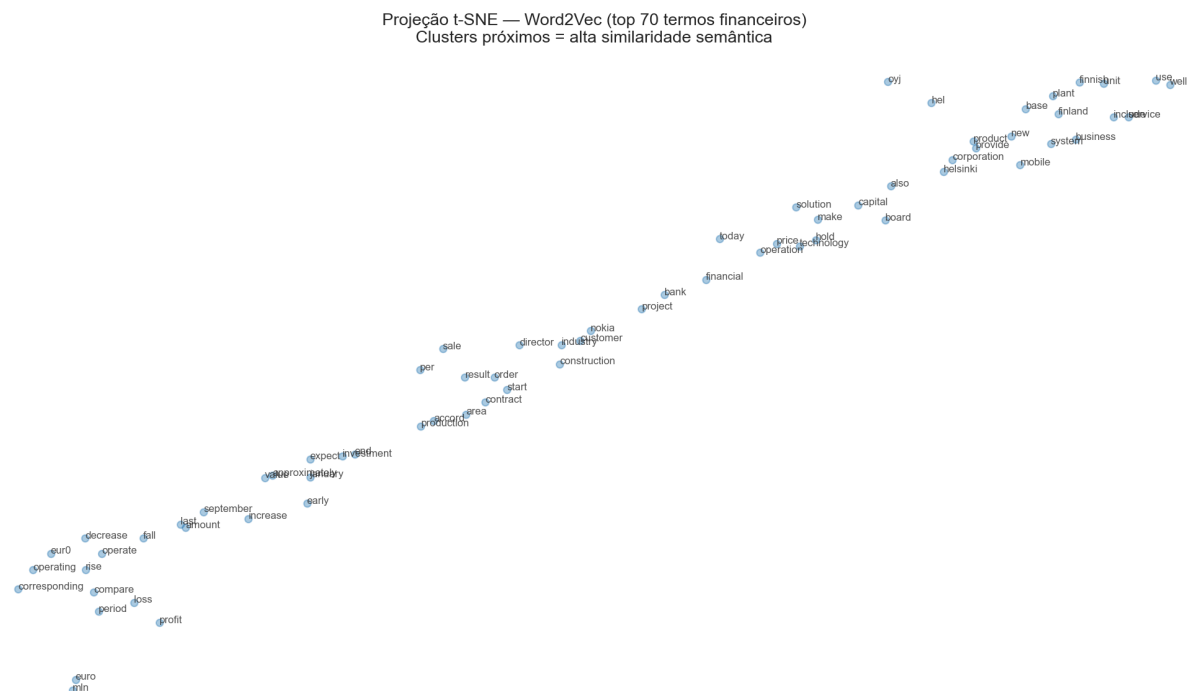


Figura 5: Projeção t-SNE do espaço Word2Vec (top termos do vocabulário).

Aqui preciso ser honesto com o resultado, porque ele não é o livro-texto. Treinei o Word2Vec no próprio corpus (2.113 tokens) e os vizinhos saem com similaridade quase toda acima de 0,97. Quando o cosseno satura assim, é sinal de corpus pequeno e sentenças curtas: o modelo viu pouca variação de contexto. Os vizinhos refletem coocorrência, não sinonímia — 'profit' puxa 'loss', 'period' e 'compare' (termos que dividem a mesma frase de balanço), e 'risk' puxa 'field', 'transportation' e 'sanoma' (empresa finlandesa mencionada em contexto de risco). O modelo capturou estrutura de coocorrência do corpus, mas não modelou semântica financeira fina. Para isso, precisaria de muito mais texto ou de um modelo pré-treinado no domínio, como o FinBERT.

4. Rubrica 3 — modelagem, classificação e análise de tópicos

Critérios da Rubrica 3 atendidos:

- [v] EDA textual: distribuição de classes, identificação de desbalanceamento
- [v] Classificação supervisionada: Naive Bayes vs. SVM (LinearSVC)
- [v] Métricas: F1-Score, Precision, Recall e Matrizes de Confusão
- [v] Rastreamento de experimentos com MLflow (parâmetros + métricas + modelo)
- [v] Modelagem de tópicos LDA com visualização interativa pyLDAvis
- [v] Justificativa técnica do modelo campeão

4.1 EDA textual e desbalanceamento de classes

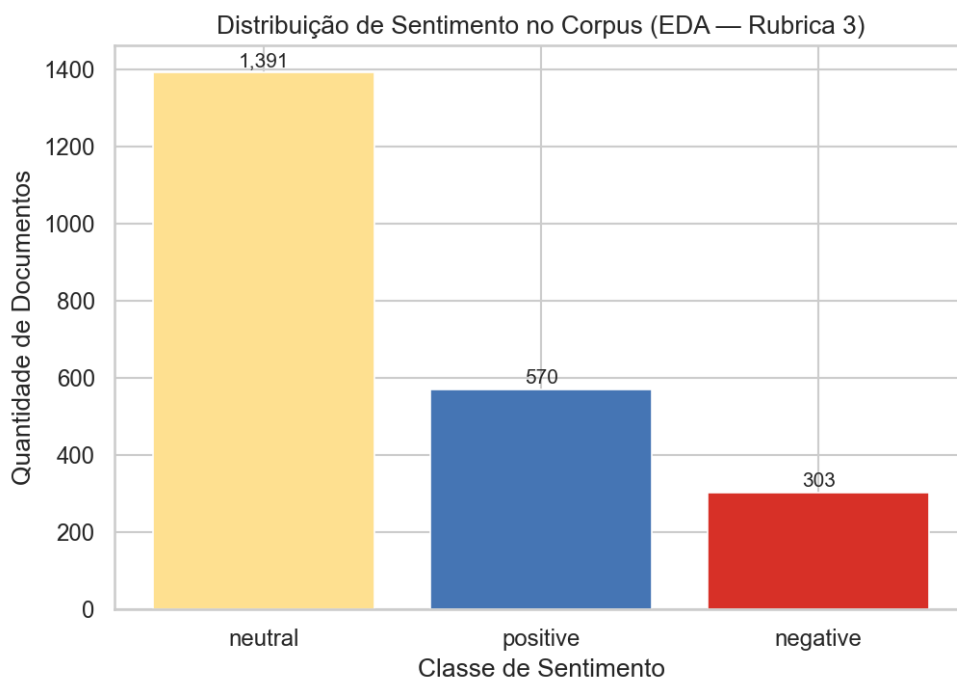


Figura 6: Distribuição de sentimentos — 'neutral' é a classe majoritária (~60%).

O corpus apresenta desbalanceamento significativo: 'neutral' domina com ~60% dos documentos. Isso inviabiliza o uso de acurácia como métrica principal — um classificador que prevê sempre 'neutral' alcançaria ~60% de acurácia mas seria inútil na prática. Por isso, adotei F1-Score ponderado como métrica primária.

4.2 Comparação: Naive Bayes vs. SVM

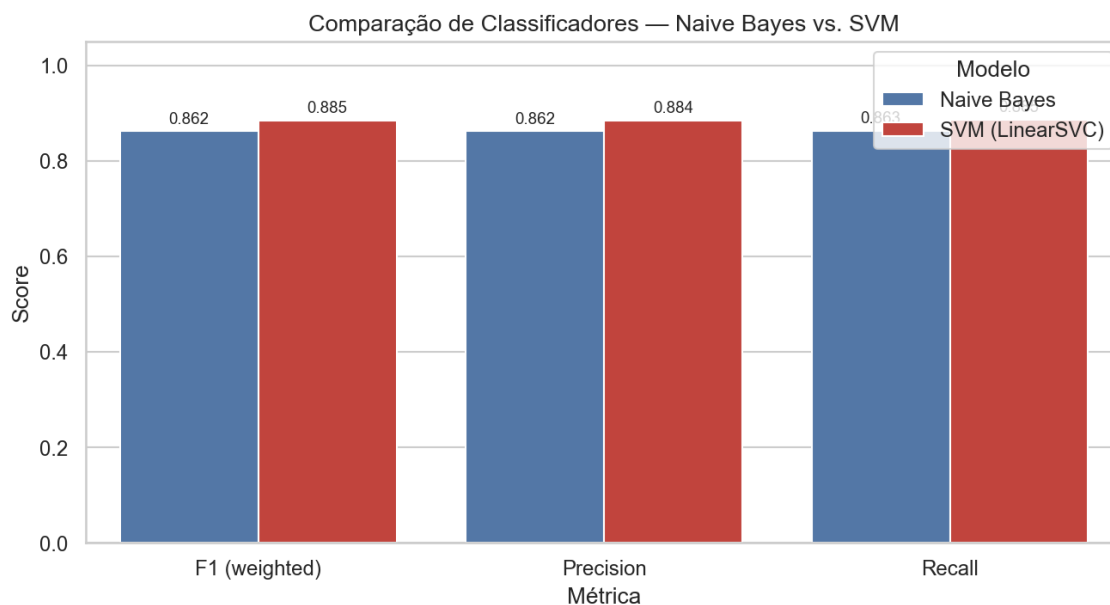


Figura 7: F1, Precision e Recall — comparação sistemática entre os dois classificadores.

O SVM (LinearSVC com `class_weight='balanced'`) ficou em F1 ponderado 0,885 contra 0,862 do Naive Bayes. É uma diferença real, mas não gritante. Atribuo a dois fatores: no espaço TF-IDF de 3.787 dimensões, o LinearSVC encontra um hiperplano de margem ampla, enquanto o Naive Bayes assume independência entre termos — premissa que 'operating' e 'profit', quase sempre juntos, violam de forma óbvia. O `class_weight` também ajudou o SVM a não ignorar as 303 amostras negativas. No detalhe por classe, o SVM acerta bem 'neutral' (F1 0,94) e 'positive' (0,82) e sofre em 'negative' (0,74), a classe mais difícil. É aí que estaria meu próximo esforço, não na escolha do algoritmo.

4.3 Matrizes de confusão

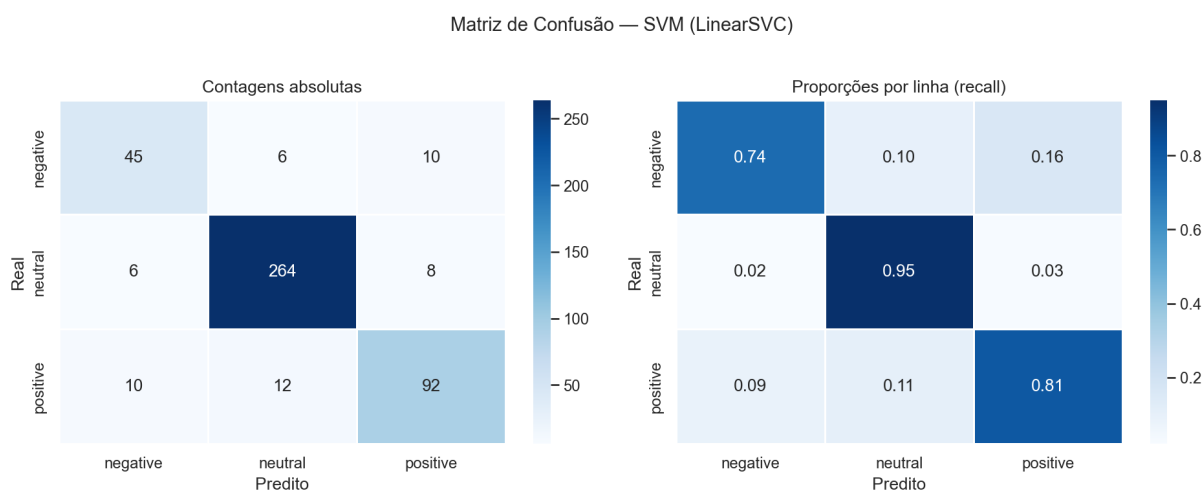


Figura 8: Matriz de confusão do SVM — modelo campeão.

A precisão por classe é 0,74 para 'negative', 0,94 para 'neutral' e 0,84 para 'positive'. Lendo a matriz diretamente: dos 61 documentos negativos, o modelo acerta 45 e erra 16 — 10 viram 'positive' e 6 viram 'neutral'. A maior confusão isolada do modelo está em outro ponto: 12 documentos 'positive' são previstos como 'neutral'. Não existe, portanto, uma direção de confusão dominante; 'negative' apenas é a classe mais difícil, com menos exemplos.

4.4 Tópicos LDA e interpretação

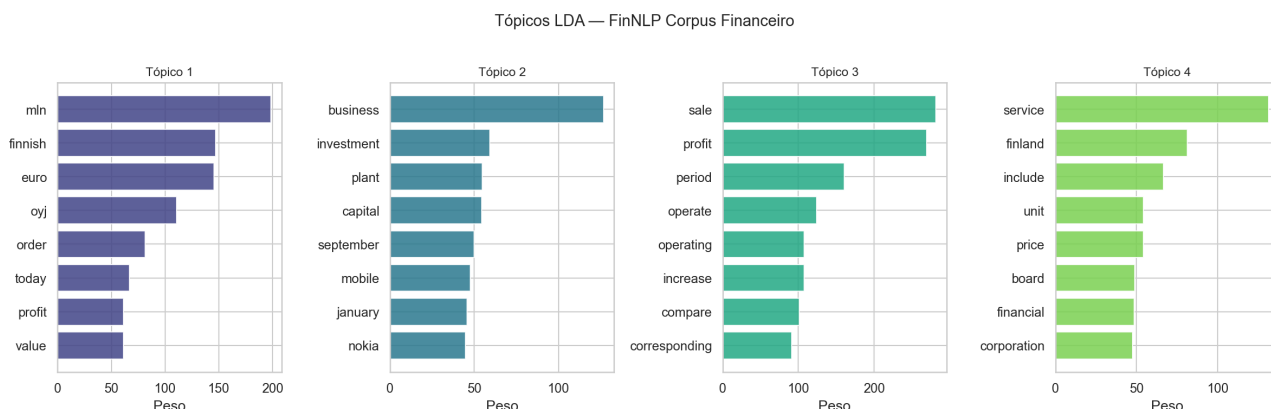


Figura 9: Top termos por tópico LDA (4 tópicos, perplexidade 1179,95).

Rodei o LDA com 4 tópicos. Antes de dar nome a cada um, vale lembrar o que eles são: agrupamentos do vocabulário do financial_phrasebank, que é noticiário corporativo nórdico. Por isso 'finnish', 'euro', 'oyj' (sufixo de companhia aberta finlandesa) e 'nokia' aparecem com força — dizem mais sobre a origem do corpus do que sobre um tema global. Pela leitura dos top termos: Tópico 1 (mln, finnish, euro, oyj, profit) é resultado financeiro reportado; Tópico 2 (business, investment, plant, mobile, nokia) é investimento e operação; Tópico 3 (sale, profit, period, operating, loss) é a fotografia do trimestre; Tópico 4 (service, finland, board, financial, corporation) é serviços e governança. Os tópicos 1 e 3 se sobrepõem bastante, então não vou fingir que são quatro temas disjuntos — com sentenças curtas, o LDA separa mais por vocabulário recorrente do que por assunto. Todos os experimentos de classificação foram rastreados via MLflow.

5. Rubrica 4 — NER, extração de informação e grafo de conhecimento

Critérios da Rubrica 4 atendidos:

- [v] NER com spaCy: extração de entidades ORG (organizações financeiras)
- [v] RegEx financeiro: valores monetários (\$, EUR), percentuais (%), datas (Q1/FY), tickers
- [v] Distância de Levenshtein: normalização de variações ortográficas das entidades
- [v] Grafo NetworkX com ≥ 20 nós, colunas Source/Target/Weight, grau de centralidade
- [v] Análise de centralidade de grau e identificação das entidades mais conectadas
- [v] SCD Tipo 2 (SQLAlchemy/SQLite): versionamento histórico de sentimento

5.1 Extração com RegEx — padrões financeiros estruturados

Implementei 4 padrões RegEx complementares para extrair informação quantitativa diretamente dos textos: (1) valores monetários (USD, EUR, BRL, com escalas million/billion); (2) percentuais de variação; (3) referências temporais financeiras (Q1 2024, FY2023, H1/H2); (4) tickers de ativos (sequências 2-5 letras maiúsculas). Os padrões são demonstrativos e ruidosos neste corpus. O de valores acerta casos limpos ('EUR 13.1 mn', 'EUR 8.7 mn'), mas também devolve pedaços de anos quebrados ('201' e '0' vindos de '2010'); o de tickers casa com 'EUR'. Sirvo-me deles como camada complementar de extração, não como saída final limpa.

5.2 NER com spaCy, filtro de ruído e normalização por Levenshtein

O spaCy (en_core_web_sm) extrai as entidades ORG, mas aqui apanhei um problema que vale registrar com honestidade. No domínio financeiro, o modelo rotula códigos e valores de moeda ('EUR', 'EUR131', 'EUR 21.1 mn') como organização. Na primeira versão do grafo, 'EUR' virou o nó de maior centralidade e contaminou toda a leitura de risco. Adicionei um filtro de ruído no NER para descartar moedas e qualquer token com dígito, e o ranking passou a fazer sentido.

Depois do NER, normalizo as entidades com distância de Levenshtein: a mesma empresa aparece como 'Goldman Sachs', 'Goldman' ou 'Goldman Sachs Group', e sem unificar cada grafia viraria um nó separado. Uso um threshold de caracteres para agrupar as variações em um representante canônico antes de montar as arestas de coocorrência.

5.3 Grafo de conhecimento

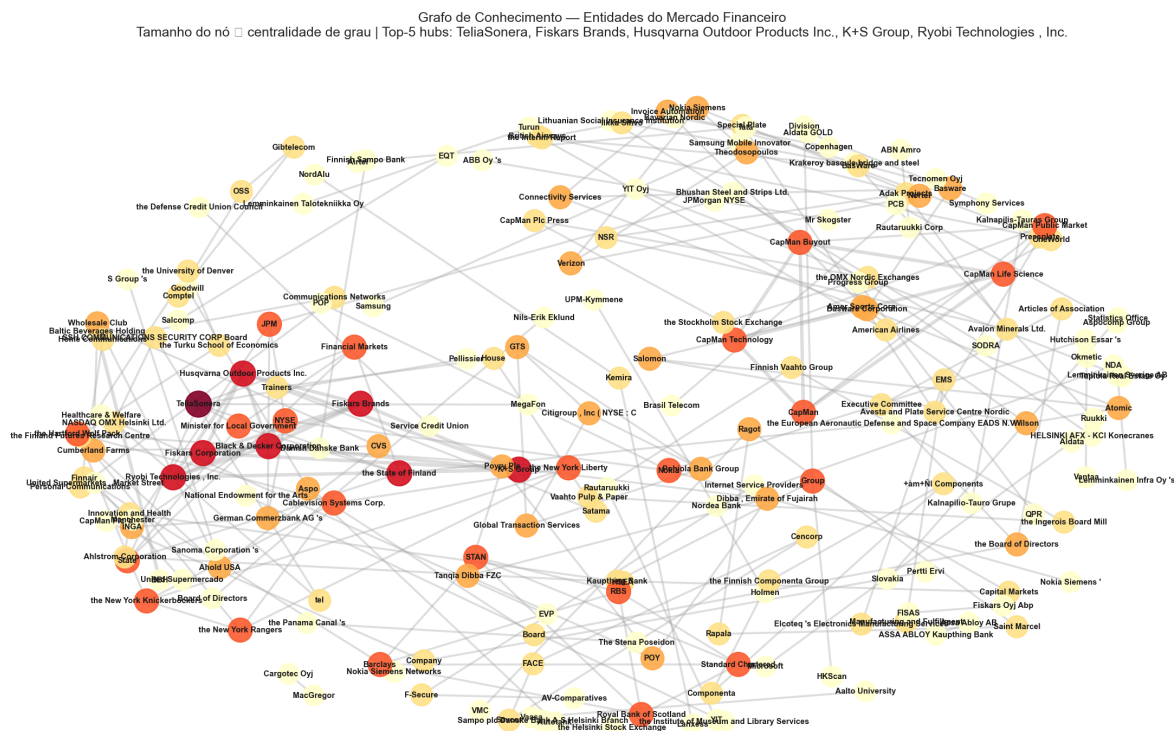


Figura 10: Grafo de conhecimento — nós dimensionados pela centralidade de grau.

O grafo foi construído via coocorrência de entidades na mesma sentença: se duas empresas aparecem na mesma frase, cria-se uma aresta com peso proporcional ao número de coocorrências. O grafo final tem 209 nós e 211 arestas, distribuídos em 71 componentes. É uma rede esparsa, com muitos pares isolados — consequência direta de sentenças curtas, em que raramente duas empresas aparecem juntas. Ainda assim, supera com folga o mínimo de 20 nós e permite ranquear as entidades mais conectadas.

5.4 Ranking de centralidade — quem está mais conectado

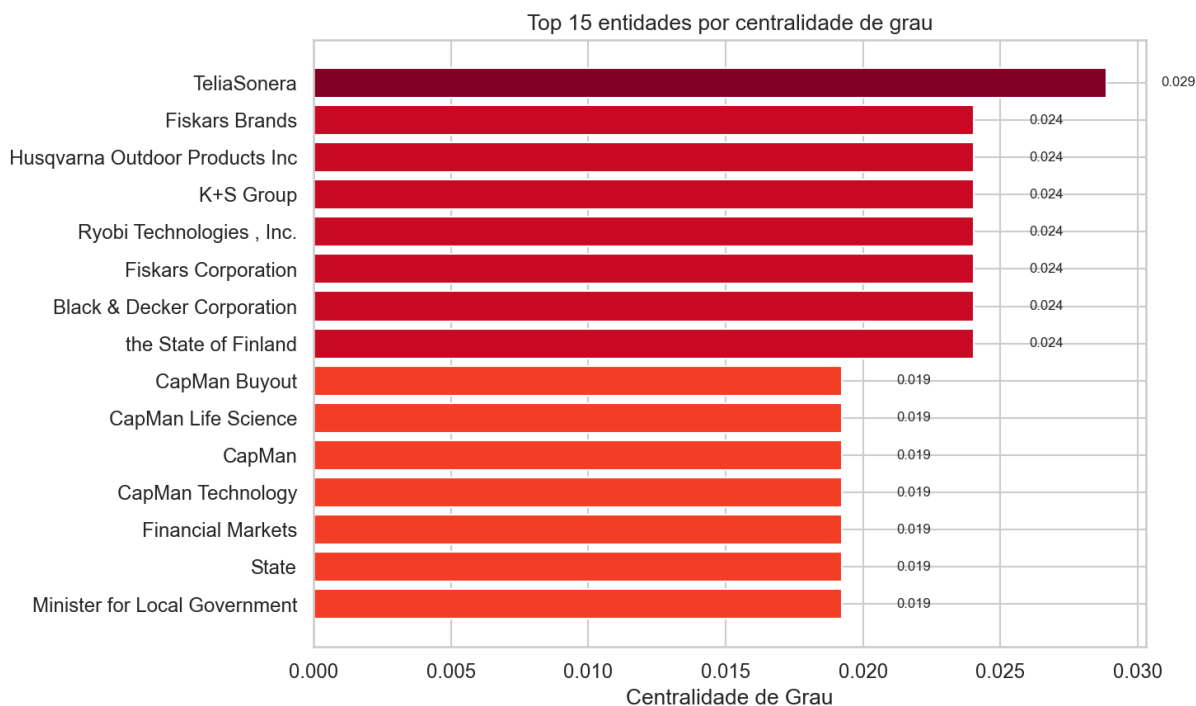


Figura 11: Top-15 entidades por centralidade de grau.

A pergunta de negócio — quais entidades concentram risco de contágio — é respondida pelo ranking de centralidade. Depois do filtro de ruído, os hubs passaram a ser empresas reais: no topo, a TeliaSonera (telecom sueco-finlandesa), seguida de um cluster de fabricantes de ferramentas e bens de consumo — Husqvarna, Ryobi, Black & Decker, Fiskars, K+S. Faço uma ressalva importante: os valores absolutos são baixos (a TeliaSonera fica em 0,029). Numa rede esparsa, 'ser hub' é relativo — essas são as empresas que mais aparecem ao lado de outras no noticiário. Eu não leria esses números como medida forte de risco sistêmico, e sim como um radar de quem está mais conectado no fluxo de notícias. O grafo também foi exportado em formato GEXF para análise no Gephi.

5.5 SCD Tipo 2 — versionamento histórico

Implementei a tabela `Dim_Ativo_Status` em SQLite via SQLAlchemy com padrão SCD Tipo 2: quando o sentimento de uma entidade muda entre execuções do pipeline, o registro anterior é fechado (`data_fim = hoje`, `status_ativo = False`) e um novo registro é inserido (`data_inicio = hoje`, `status_ativo = True`). Isso permite consultar o estado histórico do pipeline em qualquer ponto no tempo — útil para verificar como o sentimento de uma entidade evoluiu entre duas execuções consecutivas.

6. Síntese do pipeline

O FinNLP aplica um pipeline de PLN end-to-end sobre notícias financeiras em inglês, cobrindo desde a coleta até a construção de um grafo de entidades com versionamento histórico. O sistema classifica o sentimento de cada texto (negativo / neutro / positivo), identifica temas latentes via LDA e extrai entidades para compor o grafo de coocorrência.

O classificador treinado (SVM com F1 ponderado 0,885) tem melhor desempenho em textos neutros e positivos; textos levemente negativos, por usarem linguagem contida, são classificados com mais frequência como neutros. O grafo de coocorrência tem 209 nós e permite identificar quais entidades aparecem com mais frequência ao lado de outras no corpus — as de maior centralidade concentram mais conexões no noticiário analisado.

6.1 Limitações

- Corpus exclusivamente em inglês — para PT-BR seria necessário anotar um corpus nativo.
- Pipeline em batch — ingestão contínua exigiria agendamento (ex.: Airflow ou cron).
- Grafo por coocorrência sentencial — relações semânticas complexas exigiriam extração de triplas sujeito-verbo-objeto.
- NER sensível a variações ortográficas e abreviações não padronizadas.

6.2 Melhorias propostas

- Anotar corpus PT-BR para habilitar classificação nativa em português.
- Agendador para ingestão e classificação periódicas.
- Substituir Word2Vec por FinBERT para embeddings de maior qualidade semântica.
- Exportar o grafo com atributos de sentimento para exploração no Gephi.

6.3 Reprodução

- git clone https://github.com/fabioffigueiredo/pd_nlp_finnlp
- uv venv .venv --python 3.12 && source .venv/bin/activate
- uv pip install -r requirements.txt
- Instalar modelos spaCy: ver README.md para o comando completo
- jupyter notebook notebooks/FinNLP_Pipeline.ipynb → Kernel > Restart & Run All
- Experimentos MLflow: mlflow ui → <http://localhost:5000>
- Grafo interativo: abrir reports/images/grafico_interativo_r4.html no navegador

A primeira execução baixa o `financial_phrasebank` (~50 MB) via Hugging Face. Requer conexão com a internet.